practical11.java

```java
// Aim: Write a program on String Class and its methods in Java.

class StringMethodsDemo {
    static void demo() {
        String s = "  Hello, Pranay Gajbhiye!  ";
        System.out.println("trim(): [" + s.trim() + "]");
        String t = s.trim();
        System.out.println("charAt(7): " + t.charAt(7));
        System.out.println("indexOf('G'): " + t.indexOf('G'));
        System.out.println("substring(7): " + t.substring(7));
        System.out.println("toUpperCase: " + t.toUpperCase());
        System.out.println("replace 'a'->'@': " + t.replace('a','@'));
        System.out.println("contains 'Pranay': " + t.contains("Pranay"));
        String[] parts = "Mango,Banana,Orange".split(",");
        for (String p : parts) System.out.println("  " + p);
    }
}

class StringComparisonDemo {
    static void demo() {
        String s1 = "java", s3 = new String("java"), s4 = "JAVA";
        System.out.println("s1==s3: " + (s1==s3));
        System.out.println("equals: " + s1.equals(s3));
        System.out.println("equalsIgnoreCase: " + s1.equalsIgnoreCase(s4));
        System.out.println("compareTo: " + s1.compareTo(s4));
    }
}

class StringBufferDemo {
    static void demo() {
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World"); System.out.println("append: " + sb);
        sb.insert(5,","); System.out.println("insert: " + sb);
        sb.delete(5,6); System.out.println("delete: " + sb);
        sb.reverse(); System.out.println("reverse: " + sb);
    }
}

class Runner11 {
    public static void main(String[] args) {
        System.out.println("===== String Methods ====="); StringMethodsDemo.demo();
        System.out.println("\n===== Comparison =====");    StringComparisonDemo.demo();
        System.out.println("\n===== StringBuffer =====");  StringBufferDemo.demo();
    }
}
```

Output

```
===== String Methods =====
trim(): [Hello, Pranay Gajbhiye!]
charAt(7): P
indexOf('G'): 14
substring(7): Pranay Gajbhiye!
toUpperCase: HELLO, PRANAY GAJBHIYE!
replace 'a'->'@': Hello, Pr@n@y G@jbhiye!
contains 'Pranay': true
  Mango
  Banana
  Orange

===== Comparison =====
s1==s3: false
equals: true
equalsIgnoreCase: true
compareTo: 32

===== StringBuffer =====
append: Hello World
insert: Hello, World
delete: Hello World
reverse: dlroW olleH

===== StringBuilder =====
Built string: K.D.K. College | B.Tech CSE | Pranay Gajbhiye
```

Code

practical10.java

```java
// Aim: Write a program on Vector class & Array in Java.

import java.util.Vector;
import java.util.Enumeration;

class ArrayDemo {
    static void oneDimension() {
        int[] marks = {85, 92, 78, 90, 88};
        System.out.print("Marks: "); for (int m : marks) System.out.print(m + " "); System.out.println();
        int max = marks[0], sum = 0;
        for (int m : marks) { if (m > max) max = m; sum += m; }
        System.out.println("Max: " + max + " | Sum: " + sum + " | Avg: " + (double)sum/marks.length);
    }
    static void twoDimension() {
        int[][] m = {{1,2,3},{4,5,6},{7,8,9}};
        for (int[] row : m) { for (int v : row) System.out.printf("%4d",v); System.out.println(); }
        int d = 0; for (int i=0;i<m.length;i++) d += m[i][i];
        System.out.println("Diagonal sum: " + d);
    }
}

class VectorDemo {
    static void demo() {
        Vector<String> v = new Vector<>();
        v.add("Pranay"); v.add("Sita"); v.add("Raju"); v.add("Neha");
        System.out.println("Vector: " + v);
        System.out.println("get(1): " + v.get(1) + " | size: " + v.size() + " | contains Raju: " + v.contains("Raju")
        v.set(2,"Rohan"); v.remove("Sita");
        System.out.println("After set+remove: " + v);
        Enumeration<String> en = v.elements();
        while (en.hasMoreElements()) System.out.print(en.nextElement() + " ");
        System.out.println();
    }
}

class Runner10 {
    public static void main(String[] args) {
        System.out.println("===== 1D Array =====");      ArrayDemo.oneDimension();
        System.out.println("\n===== 2D Array =====");     ArrayDemo.twoDimension();
        System.out.println("\n===== Vector Class =====");  VectorDemo.demo();
        System.out.println("\nArray: fixed size, fast. Vector: dynamic, synchronized.");
    }
}
```

Output

```
===== 1D Array =====
Marks: 85 92 78 90 88
Max: 92 | Sum: 433 | Avg: 86.6

===== 2D Array =====
   1   2   3
   4   5   6
   7   8   9
Diagonal sum: 15

===== Vector Class =====
Vector: [Pranay, Sita, Raju, Neha]
get(1): Sita | size: 4 | contains Raju: true
After set+remove: [Pranay, Rohan, Neha]
Pranay Rohan Neha

Array: fixed size, fast. Vector: dynamic, synchronized.
```

Code

practical9.java

```java
// Aim: Write a program on Multi-Threading and use various methods of Thread class in Java.

class MyThread extends Thread {
    private String taskName;
    MyThread(String name, String task) { super(name); this.taskName = task; }
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + " | " + taskName + " | Step " + i);
            try { Thread.sleep(300); } catch (InterruptedException e) { }
        }
    }
}

class MyRunnable implements Runnable {
    private String name;
    MyRunnable(String n) { name = n; }
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Runnable-" + name + " | Step " + i);
            try { Thread.sleep(200); } catch (InterruptedException e) { }
        }
    }
}

class SharedCounter {
    private int count = 0;
    synchronized void increment() { count++; }
    int getCount() { return count; }
}

class Runner9 {
    public static void main(String[] args) throws InterruptedException {
        System.out.println("===== Extending Thread =====");
        MyThread t1 = new MyThread("T1","Download"), t2 = new MyThread("T2","Upload");
        t1.setPriority(Thread.MAX_PRIORITY); t2.setPriority(Thread.MIN_PRIORITY);
        t1.start(); t2.start(); t1.join(); t2.join();

        System.out.println("\n===== Runnable Interface =====");
        Thread r1 = new Thread(new MyRunnable("Alpha")), r2 = new Thread(new MyRunnable("Beta"));
        r1.start(); r2.start(); r1.join(); r2.join();

        System.out.println("\n===== Synchronization =====");
        SharedCounter c = new SharedCounter();
        Thread i1 = new Thread(() -> { for(int i=0;i<1000;i++) c.increment(); });
        Thread i2 = new Thread(() -> { for(int i=0;i<1000;i++) c.increment(); });
        i1.start(); i2.start(); i1.join(); i2.join();
        System.out.println("Final count (expected 2000): " + c.getCount());

        Thread cur = Thread.currentThread();
        System.out.println("\nCurrent thread: " + cur.getName() + " | Priority: " + cur.getPriority() + " | State: "
    }
}
```

Output

```
===== Extending Thread =====
T1 | Download | Step 1
T2 | Upload | Step 1
T1 | Download | Step 2
T2 | Upload | Step 2
T1 | Download | Step 3
T2 | Upload | Step 3

===== Runnable Interface =====
Runnable-Alpha | Step 1
Runnable-Beta | Step 1
Runnable-Alpha | Step 2
Runnable-Beta | Step 2
Runnable-Alpha | Step 3
Runnable-Beta | Step 3

===== Synchronization =====
Final count (expected 2000): 2000

Current thread: main | Priority: 5 | State: RUNNABLE
```

Code

mypackage/MathHelper.java

```
package mypackage;

public class MathHelper {
    public int    add(int a, int b)          { return a + b; }
    public int    subtract(int a, int b)     { return a - b; }
    public int    multiply(int a, int b)     { return a * b; }
    public double divide(double a, double b) { if (b == 0) { System.out.println("Error: Division by zero."); return 6
    public boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i * i <= n; i++) if (n % i == 0) return false;
        return true;
    }
}
```

practical8.java

```
// Aim: Write a program on Packages (pre-defined & user-defined packages) in Java.

import java.util.Scanner;
import java.util.ArrayList;
import java.util.Arrays;
import mypackage.MathHelper;

class Runner8 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: "); String name = sc.nextLine();
        System.out.print("Enter your age : "); int age = sc.nextInt();
        System.out.println("Hello, " + name + "! You are " + age + " years old.");

        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Mango"); fruits.add("Banana"); fruits.add("Orange"); fruits.add("Apple");
        System.out.println("Fruits: " + fruits);
        fruits.remove("Banana");
        System.out.println("After remove: " + fruits);

        int[] numbers = {5, 3, 8, 1, 9, 2, 7};
        Arrays.sort(numbers);
        System.out.println("Sorted: " + Arrays.toString(numbers));

        MathHelper mh = new MathHelper();
        System.out.println("Add(10,5)=" + mh.add(10,5) + " isPrime(7)=" + mh.isPrime(7));
        sc.close();
    }
}
```

Output

```
Enter your name: Pranay
Enter your age : 20
Hello, Pranay! You are 20 years old.

===== Pre-defined Package: java.util.ArrayList =====
Fruits list : [Mango, Banana, Orange, Apple]
After remove: [Mango, Orange, Apple]
Size        : 3

===== Pre-defined Package: java.util.Arrays =====
Before sort: [5, 3, 8, 1, 9, 2, 7]
After sort : [1, 2, 3, 5, 7, 8, 9]
Search 7   : index 4

===== User-defined Package: mypackage.MathHelper =====
Add(10, 5)      : 15
Subtract(10, 5) : 5
Multiply(4, 6)  : 24
Divide(10, 4)   : 2.5
isPrime(7)      : true
isPrime(10)     : false
```

practical7.java

```java
// Aim: Write a program to create a User Defined Exception in Java.

// Checked custom exception
class InsufficientFundsException extends Exception {
    private double amount;
    InsufficientFundsException(double amt) { super("Insufficient funds! Short by: Rs. " + amt); this.amount = amt; }
    double getAmount() { return amount; }
}

// Unchecked custom exception
class InvalidAgeException extends RuntimeException {
    InvalidAgeException(int age) { super("Invalid age: " + age + ". Age must be between 0 and 150."); }
}

class BankAccount {
    private String owner; private double balance;
    BankAccount(String o, double b) { owner = o; balance = b; }
    void deposit(double a) { balance += a; System.out.println("Deposited Rs. " + a + " | Balance: Rs. " + balance); }
    void withdraw(double a) throws InsufficientFundsException {
        if (a > balance) throw new InsufficientFundsException(a - balance);
        balance -= a; System.out.println("Withdrawn Rs. " + a + " | Balance: Rs. " + balance);
    }
    double getBalance() { return balance; }
}

class VoterRegistration {
    static void register(String name, int age) {
        if (age < 18 || age > 150) throw new InvalidAgeException(age);
        System.out.println(name + " (age " + age + ") registered as voter successfully.");
    }
}

class Runner7 {
    public static void main(String[] args) {
        System.out.println("===== Checked Custom Exception =====");
        BankAccount acc = new BankAccount("Pranay", 5000.0);
        acc.deposit(2000.0);
        try { acc.withdraw(3000.0); acc.withdraw(10000.0); }
        catch (InsufficientFundsException e) {
            System.out.println("Exception: " + e.getMessage());
            System.out.println("Short by: Rs. " + e.getAmount());
        }
        System.out.println("Final Balance: Rs. " + acc.getBalance());

        System.out.println("\n===== Unchecked Custom Exception =====");
        VoterRegistration.register("Pranay", 20);
        VoterRegistration.register("Sita", 35);
        try { VoterRegistration.register("Raju", 15); }
        catch (InvalidAgeException e) { System.out.println("Exception: " + e.getMessage()); }
        try { VoterRegistration.register("Ghost", 200); }
        catch (InvalidAgeException e) { System.out.println("Exception: " + e.getMessage()); }
    }
}
```

Output

```
===== Checked Custom Exception =====
Deposited Rs. 2000.0 | Balance: Rs. 7000.0
Withdrawn Rs. 3000.0 | Balance: Rs. 4000.0
Exception: Insufficient funds! Short by: Rs. 6000.0
Short by: Rs. 6000.0
Final Balance: Rs. 4000.0

===== Unchecked Custom Exception =====
Pranay (age 20) registered as voter successfully.
Sita (age 35) registered as voter successfully.
Exception: Invalid age: 15. Age must be between 0 and 150.
Exception: Invalid age: 200. Age must be between 0 and 150.
```

practical6.java

```java
// Aim: Write a program to implement the concepts of Exception Handling in Java.

import java.io.*;

class BasicException {
    static void demo() {
        try { int r = 10 / 0; }
        catch (ArithmeticException e) { System.out.println("Caught ArithmeticException: " + e.getMessage()); }
        finally { System.out.println("finally block always executes."); }
    }
}

class MultipleCatch {
    static void demo() {
        try { int[] a = {1,2,3}; System.out.println(a[5]); }
        catch (ArrayIndexOutOfBoundsException e) { System.out.println("Caught AIOOBE: " + e.getMessage()); }
        try { Integer.parseInt("abc"); }
        catch (NumberFormatException e) { System.out.println("Caught NFE: " + e.getMessage()); }
        try { String s = null; s.length(); }
        catch (NullPointerException e) { System.out.println("Caught NPE: " + e.getMessage()); }
    }
}

class FileReader6 {
    static void readFile(String f) throws IOException { throw new IOException("File not found: " + f); }
    static void demo() {
        try { readFile("data.txt"); }
        catch (IOException e) { System.out.println("Caught IOException: " + e.getMessage()); }
    }
}

class NestedTryCatch {
    static void demo() {
        try {
            try { int[] a = new int[2]; a[5] = 10; }
            catch (ArrayIndexOutOfBoundsException e) { System.out.println("Inner catch: " + e.getMessage()); }
            int r = 10 / 0;
        } catch (ArithmeticException e) { System.out.println("Outer catch: " + e.getMessage()); }
        finally { System.out.println("Outer finally block."); }
    }
}

class Runner6 {
    public static void main(String[] args) {
        System.out.println("--- try-catch-finally ---");   BasicException.demo();
        System.out.println("\n--- Multiple catch ---");     MultipleCatch.demo();
        System.out.println("\n--- Checked Exception ---");  FileReader6.demo();
        System.out.println("\n--- Nested try-catch ---");   NestedTryCatch.demo();
    }
}
```

Output

```
--- try-catch-finally ---
Caught ArithmeticException: / by zero
finally block always executes.

--- Multiple catch ---
Caught AIOOBE: Index 5 out of bounds for length 3
Caught NFE: For input string: "abc"
Caught NPE: Cannot invoke "String.length()" because "s" is null

--- Checked Exception ---
Caught IOException: File not found: data.txt

--- Nested try-catch ---
Inner catch: Index 5 out of bounds for length 2
Outer catch: / by zero
Outer finally block.
```

practical5.java

```java
// Aim: Write a program to demonstrate Inheritance and its types in Java.

// 1. Single Inheritance
class Person {
    String name; int age;
    Person(String n, int a) { name = n; age = a; }
    void introduce() { System.out.println("Hi, I am " + name + " and I am " + age + " years old."); }
}
class Student5 extends Person {
    int rollNo;
    Student5(String n, int a, int r) { super(n, a); rollNo = r; }
    void introduce() { super.introduce(); System.out.println("My Roll No is: " + rollNo); }
}

// 2. Multilevel Inheritance
class LivingBeing { void breathe() { System.out.println("Living beings breathe."); } }
class Mammal extends LivingBeing { void feedMilk() { System.out.println("Mammals feed milk to offspring."); } }
class Dog extends Mammal { void bark() { System.out.println("Dog barks: Woof!"); } }

// 3. Hierarchical Inheritance
class Shape5 { double area() { return 0; } }
class Circle5 extends Shape5 {
    double radius; Circle5(double r) { radius = r; }
    @Override double area() { return Math.PI * radius * radius; }
}
class Rectangle5 extends Shape5 {
    double length, breadth; Rectangle5(double l, double b) { length = l; breadth = b; }
    @Override double area() { return length * breadth; }
}
class Triangle5 extends Shape5 {
    double base, height; Triangle5(double b, double h) { base = b; height = h; }
    @Override double area() { return 0.5 * base * height; }
}

// 4. Multiple Inheritance (interfaces)
interface Flyable { void fly(); }
interface Swimmable { void swim(); }
class Duck implements Flyable, Swimmable {
    public void fly()  { System.out.println("Duck is flying."); }
    public void swim() { System.out.println("Duck is swimming."); }
    void quack() { System.out.println("Duck says: Quack!"); }
}

class Runner5 {
    public static void main(String[] args) {
        System.out.println("===== 1. Single Inheritance =====");
        new Student5("Pranay", 20, 42).introduce();

        System.out.println("\n===== 2. Multilevel Inheritance =====");
        Dog d = new Dog(); d.breathe(); d.feedMilk(); d.bark();

        System.out.println("\n===== 3. Hierarchical Inheritance =====");
        System.out.printf("Circle    area: %.4f%n", new Circle5(5).area());
        System.out.printf("Rectangle area: %.4f%n", new Rectangle5(4, 6).area());
        System.out.printf("Triangle  area: %.4f%n", new Triangle5(3, 8).area());

        System.out.println("\n===== 4. Multiple Inheritance (interfaces) =====");
        Duck duck = new Duck(); duck.fly(); duck.swim(); duck.quack();
    }
}
```

Output

```
===== 1. Single Inheritance =====
Hi, I am Pranay and I am 20 years old.
My Roll No is: 42

===== 2. Multilevel Inheritance =====
Living beings breathe.
Mammals feed milk to offspring.
Dog barks: Woof!

===== 3. Hierarchical Inheritance =====
Circle    area: 78.5398
Rectangle area: 24.0000
Triangle  area: 12.0000

===== 4. Multiple Inheritance (interfaces) =====
Duck is flying.
Duck is swimming.
Duck says: Quack!
```

practical3.java

```java
// Aim: Write a program to demonstrate Method Overloading in Java.

class MathUtils {
    public double area(double radius)           { return Math.PI * radius * radius; }
    public double area(double length, double b) { return length * b; }
    public double area(int side)               { return (double) side * side; }

    public double perimeter(double a, double b, double c) { return a + b + c; }
    public double perimeter(double length, double breadth) { return 2 * (length + breadth); }
    public double perimeter(int side)          { return 4.0 * side; }
}

class Printer {
    public void print(int value)    { System.out.println("Integer value : " + value); }
    public void print(double value) { System.out.println("Double value  : " + value); }
    public void print(String value) { System.out.println("String value  : " + value); }
    public void print(boolean value){ System.out.println("Boolean value : " + value); }
}

class Runner3 {
    public static void main(String[] args) {
        MathUtils m = new MathUtils();

        System.out.println("===== Area Calculations =====");
        System.out.printf("Area of circle (r=5)           : %.4f%n", m.area(5.0));
        System.out.printf("Area of rectangle (4 x 6)      : %.4f%n", m.area(4.0, 6.0));
        System.out.printf("Area of square (side=7)         : %.4f%n", m.area(7));

        System.out.println("\n===== Perimeter Calculations =====");
        System.out.printf("Perimeter of triangle (3,4,5)   : %.4f%n", m.perimeter(3.0, 4.0, 5.0));
        System.out.printf("Perimeter of rectangle (4 x 6)  : %.4f%n", m.perimeter(4.0, 6.0));
        System.out.printf("Perimeter of square (side=7)     : %.4f%n", m.perimeter(7));

        System.out.println("\n===== Printer — Overloading by Type =====");
        Printer p = new Printer();
        p.print(42);
        p.print(3.14);
        p.print("Hello, Pranay!");
        p.print(true);
    }
}
```

Output

```
===== Area Calculations =====
Area of circle (r=5)           : 78.5398
Area of rectangle (4 x 6)      : 24.0000
Area of square (side=7)         : 49.0000

===== Perimeter Calculations =====
Perimeter of triangle (3,4,5)   : 12.0000
Perimeter of rectangle (4 x 6)  : 20.0000
Perimeter of square (side=7)     : 28.0000

===== Printer — Overloading by Type =====
Integer value : 42
Double value  : 3.14
String value  : Hello, Pranay!
Boolean value : true
```

Code

practical4.java

```java
// Aim: Write a program to demonstrate the use of keywords static, final, and this in Java.

class Counter {
    static int count = 0;
    Counter() { count++; }
    static void displayCount() { System.out.println("Total objects created: " + count); }
}

class MathConstants {
    final double PI = 3.14159265358979;
    final double circleArea(double r) { return PI * r * r; }
}

final class Immutable {
    final int value;
    Immutable(int v) { this.value = v; }
    void show() { System.out.println("Immutable value: " + value); }
}

class Employee {
    String name; int id; double salary;
    Employee() { this("Unknown", 0, 0.0); System.out.println("Default Employee created."); }
    Employee(String name, int id, double salary) { this.name = name; this.id = id; this.salary = salary; }
    void display() { System.out.println("ID: " + this.id + " | Name: " + this.name + " | Salary: " + this.salary); }
    Employee getSelf() { return this; }
}

class Runner4 {
    public static void main(String[] args) {
        System.out.println("===== static Keyword =====");
        new Counter(); new Counter(); new Counter();
        Counter.displayCount();

        System.out.println("\n===== final Keyword =====");
        MathConstants mc = new MathConstants();
        System.out.println("PI = " + mc.PI);
        System.out.printf("Area of circle (r=7): %.4f%n", mc.circleArea(7));
        new Immutable(100).show();

        System.out.println("\n===== this Keyword =====");
        Employee e1 = new Employee("Pranay", 101, 45000.0);
        e1.display();
        Employee e2 = new Employee();
        e2.display();
        System.out.println("getSelf() returns same object: " + (e1.getSelf() == e1));
    }
}
```

Output

```
===== static Keyword =====
Total objects created: 3

===== final Keyword =====
PI = 3.14159265358979
Area of circle (r=7): 153.9380
Immutable value: 100

===== this Keyword =====
ID: 101 | Name: Pranay | Salary: 45000.0
Default Employee created.
ID: 0 | Name: Unknown | Salary: 0.0
getSelf() returns same object: true
```

Code

practical2.java

```java
// Aim: Write a program to demonstrate Constructor and Types of Constructor in Java.

class Box {
    double length, breadth, height;

    // 1. Default Constructor
    Box() {
        length = 1.0; breadth = 1.0; height = 1.0;
        System.out.println("Default Constructor called.");
    }

    // 2. Parameterized Constructor
    Box(double l, double b, double h) {
        length = l; breadth = b; height = h;
        System.out.println("Parameterized Constructor called.");
    }

    // 3. Copy Constructor
    Box(Box other) {
        this.length = other.length; this.breadth = other.breadth; this.height = other.height;
        System.out.println("Copy Constructor called.");
    }

    // 4. Constructor Overloading — Cube
    Box(double side) {
        length = side; breadth = side; height = side;
        System.out.println("Cube Constructor called.");
    }

    double volume() { return length * breadth * height; }

    void display() {
        System.out.println("Length=" + length + ", Breadth=" + breadth + ", Height=" + height);
        System.out.println("Volume = " + volume());
    }
}

class Runner2 {
    public static void main(String[] args) {
        System.out.println("===== 1. Default Constructor =====");
        Box b1 = new Box();
        b1.display();

        System.out.println("\n===== 2. Parameterized Constructor =====");
        Box b2 = new Box(4.0, 3.0, 2.0);
        b2.display();

        System.out.println("\n===== 3. Copy Constructor =====");
        Box b3 = new Box(b2);
        b3.display();

        System.out.println("\n===== 4. Constructor Overloading — Cube =====");
        Box b4 = new Box(5.0);
        b4.display();
    }
}
```

Output

```
===== 1. Default Constructor =====
Default Constructor called.
Length=1.0, Breadth=1.0, Height=1.0
Volume = 1.0

===== 2. Parameterized Constructor =====
Parameterized Constructor called.
Length=4.0, Breadth=3.0, Height=2.0
Volume = 24.0

===== 3. Copy Constructor =====
Copy Constructor called.
Length=4.0, Breadth=3.0, Height=2.0
Volume = 24.0

===== 4. Constructor Overloading — Cube =====
Cube Constructor called.
Length=5.0, Breadth=5.0, Height=5.0
Volume = 125.0
```

Code

Product.java

```java
// Aim: write a program to Demonstrate method overloading in Java

import java.io.*;
class Product
{
    public int multiply(int a, int b) {return a * b;} // Multiplying 2 integer values
    public int multiply(int a, int b, int c) {return a * b * c;} // Multiplying 3 integer values
    public int prod(int a, int b, int c) {return a * b * c;} // Product of 3 integers
    public double prod(double a, double b, double c) {return a * b * c;} // Product of 3 doubles
}

class Student {
    // Student ID with name first
    public void studentId(String name, int rollNo) {System.out.println("Name: " + name + ", Roll-No: " + rollNo);}
    // Student ID with rollNo first
    public void studentId(int rollNo, String name) {System.out.println("Roll-No: " + rollNo + ", Name: " + name);}
}
class Runner {
    public static void main(String[] args) {

        Product p = new Product();
        int prod1 = p.multiply(1, 2);  // Multiply 2 numbers
        System.out.println("Product of two integers: " + prod1);
        int prod2 = p.multiply(1, 2, 3);  // Multiply 3 numbers
        System.out.println("Product of three integers: " + prod2);
        System.out.println("Product of three integers: " + p.prod(1, 2, 3)); // Product of integers
        System.out.println("Product of three doubles: " + p.prod(1.0, 2.0, 3.0)); // Product of doubles
        Student s = new Student();
        s.studentId("Sweta", 1);
        s.studentId(2, "Gudly");
    }}
```

Calculator.java

```java
import java.util.*;
class Calculator{
    public static void main(String []agrs)
    {   Scanner scanner = new Scanner(System.in);
        System.out.println("Pranay is trying to make a Calculator!!");
        System.out.println("========JAVA CALCULATOR INTERFACE========");
        System.out.println("Enter First Digit");
        int FirstDigit = scanner.nextInt();
        System.out.println("Enter Second Digit");
        int SecondDigit = scanner.nextInt();
        scanner.nextLine();
        System.out.println("Please Broh!! Enter YOur Name");
        String Name = scanner.nextLine();
        int  sum = FirstDigit + SecondDigit;
        System.out.println("hellow, " + Name + " the Sum of "  + FirstDigit + " And " + SecondDigit + " is " + sum);
        scanner.close();
    }
}
```

Output

```
Product of two integers: 2
Product of three integers: 6
Product of three integers: 6
Product of three doubles: 6.0
Name: Sweta, Roll-No: 1
Roll-No: 2, Name: Gudly
```